

Working with Context Files

Ground every refinement in the files, repos, and docs that actually matter.

3-5 pages

Printable web guide

Accessible figure notes included

What it does and why it matters

Generic prompts fail when a task depends on your specific product, architecture, or constraints. Repo context gives the model a cleaner frame before it rewrites the request.

Leave knowing which files to attach, how to avoid noise, and how to validate the improvement from repo context.

Step-by-step workflow

STEP 1

Choose context that explains the task, not the whole repo

Use Upload files for surgical context, Upload folder for small batches, or Connect local Git when you want RyFine to respect ignores and skip obvious noise.



ANNOTATED RYFINE WALKTHROUGH

PRIMARY ACTION

Use Upload files for surgical context, Upload folder for small batches, or Connect local Git when you want RyFine to respect ignores and skip obvious noise.

ANNOTATION

Context is strongest when it explains the system boundary, target files, and constraints without flooding the model.

Alt text: A repo context drawer with buttons for Upload files, Upload folder, and Connect local Git above a list of selected files.

How to use the repo context drawer: The drawer is located at the bottom of the screen and contains three buttons: Upload files, Upload folder, and Connect local Git. The drawer is open by default and shows a list of selected files.

STEP 2

Preview and include only the files you want sent

Use the preview controls to confirm the selected content and keep only the snippets that are worth spending tokens on. RyFine shows the included count and selected character total.



ANNOTATED RYFINE WALKTHROUGH

PRIMARY ACTION

Use the preview controls to confirm the selected content and keep only the snippets that are worth spending tokens on. RyFine shows the included count and selected character total.

ANNOTATION

The include checkbox is your relevance filter. Treat it like an editorial step.

Alt text: A list of uploaded repo files with include checkboxes, character counts, and preview buttons for each entry.

Alt text: A list of uploaded repo files with include checkboxes, character counts, and preview buttons for each entry.

STEP 3

Run both versions when you want proof that context helped

Use the compare flow to run the prompt once without repo context and once with it. The context-aware result should mention the real modules, architecture, or workflow expectations captured in your files.



ANNOTATED RYFINE WALKTHROUGH

PRIMARY ACTION

Use the compare flow to run the prompt once without repo context and once with it. The context-aware result should mention the real modules, architecture, or workflow expectations captured in your files.

ANNOTATION

This is the easiest way to justify the extra setup to yourself or a team.

Alt text: Two result panels labeled baseline and with repo context, each containing a refined coding prompt with different levels of specificity.

Two result panels labeled baseline and with repo context, each containing a refined coding prompt with different levels of specificity.

Call to action

Try RyFine now! Open the app, run one prompt with the workflow above, and save the version you would want to reuse.

SUMMARY

Quick brief

Covers file upload, local Git ingestion, GitHub import, and comparing a refinement with and without repo context.

USE CASES

Where this guide helps most

- Generate implementation prompts grounded in an existing codebase.
- Write migration plans tied to real package versions and file structure.
- Refine onboarding, review, or documentation prompts against live project files.

BEST PRACTICES

Keep the workflow tight

- Favor small, high-signal file sets over complete repository dumps.
- Use preview before every important run.
- Compare with and without context when refining prompts for a new workflow.